SISTEMAS OPERATIVOS

Aula Teórica 7

Daniel Castro
2025/2026

UAlg
UNIVERSIDADE DO ALGARVE

How one process can pass information to another?

Making sure two or more processes or threads do not get in each's way,
Ex. Two threads in an airline reservation system, each trying to grab the last seat on a plane for different customers.

Proper sequencing when dependencies are present,
Ex. if thread A produces data and thread B prints them, B as to wait until A has produced some more data before starting to print.

1. Race Conditions
2. Critical Regions
3. Mutual Exclusion with Busy Waiting
4. Sleep and Wakeu
5. Semaphores

---

6. Mutexes
7. Monitors
8. Message Passing
9. Barriers
10. Priority Inversion
11. Avoiding Locks: Read-Copy-Update

6. Mutexes (1)

Mutexes: Simplified version of a semaphore.

Mutex is a shared variable with two possible states: **unlocked** (0) or **locked** (any value >0)

When a thread needs access to the critical region, it calls *mutex_lock*
- If *mutex==0* the thread is free to go;
- Otherwise if mutex>0 the calling thread is blocked until the thread in the critical region is finished and calls *mutex_unlock*.

They are so simple that they can easily be implemented in the user space , with TSL or XCHG.

```
mutex_lock:
    TSL REGISTER,MUTEX          | copy mutex to register and set mutex to 1
    CMP REGISTER,#0             | was mutex zero?
    JZE ok                      | if it was zero, mutex was unlocked, so return
    CALL thread_yield           | mutex is busy; schedule another thread
    JMP mutex_lock              | try again later
ok: RET | return to caller; critical region entered

mutex_unlock:
    MOVE MUTEX,#0               | store a 0 in mutex
    RET | return to caller
```

Implementation of *mutex_lock* and *mutex_unlock*

6. Mutexes (2)

```
enter_region:
    TSL REGISTER,LOCK    | copy lock to register and set lock to 1
    CMP REGISTER,#0      | was lock zero?
    JNE enter_region     | if it was not zero, lock was set, so loop
    RET                  | return to caller; critical region entered

leave_region:
    MOVE LOCK,#0         | store a 0 in lock
    RET                  | return to caller
```

Figure 2-25. Entering and leaving a critical region using the TSL instruction.

```
mutex_lock:
    TSL REGISTER,MUTEX   | copy mutex to register and set mutex to 1
    CMP REGISTER,#0      | was mutex zero?
    JZE ok               | if it was zero, mutex was unlocked, so return
    CALL thread_yield    | mutex is busy; schedule another thread
    JMP mutex_lock       | try again later
ok: RET | return to caller; critical region entered

mutex_unlock:
    MOVE MUTEX,#0        | store a 0 in mutex
    RET | return to caller
```

Implementation of *mutex_lock* and *mutex_unlock*

The code of *mutex_lock* is similar to the code of *enter_region*.

When *enter_region* fails to enter the critical region, keeps testing the lock repeatedly (busy waiting).
Sooner or later the process holding the lock gets run and release it.
But, with (user) threads the situation is different because there is no clock that stops a thread that have run too long.
Consequently, a thread that tries to adquire a lock by busy waiting will loop forever and never adquire the lock.

With mutexes there is no busy waiting.
Because when the later thread fails to adquire the lock, it calls *thread_yield* to give the CPU to another thread.

Thread_yield is just a call to the thread schedular in user space, it is very fast.
As a consequence, neither *mutex_lock* nor *mutex_unlock* requires any kernel calls.

6. Mutexes (3)

mutex_trilock - call in some thread packages that either acquires the lock or return a code for failure. This call gives the thread the flexibility to decide what to do next, if there are alternatives to just waiting.

futex – fast user space mutex
A futex is a feature of linux that implements basic locking (much like a mutex) but avoids dropping into the kernel unless it really as to. Since switching to the kernel and back is quite expensive, doing so, improves performance considerably.

Additionally to mutexes, Pthread offers a second synchronazation mechanism – condition variables. Condition variables allow threads to block due to some condition not being met.

Condition variables and mutexes are always used together.

Thread call	Description
pthread_mutex_init	Create a mutex
pthread_mutex_destroy	Destroy an existing mutex
pthread_mutex_lock	Acquire a lock or block
pthread_mutex_trylock	Acquire a lock or fail
pthread_mutex_unlock	Release a lock

Figure 2-31. Some of the Pthreads' calls relating to mutexes.

Thread call	Description
pthread_cond_init	Create a condition variable
pthread_cond_destroy	Destroy a condition variable
pthread_cond_wait	Block waiting for a signal
pthread_cond_signal	Signal another thread and wake it up
pthread_cond_broadcast	Signal multiple threads and wake all of them

Figure 2-32. Some of the Pthreads calls relating to condition variables.

Condition variables (unlike semaphores) have no memory. So, if a signal is sent to a conditional variable on which no thread is waiting, the signal is lost.

6. Mutexes (4)

PRODUCER – CONSUMER PROBLEM

An example of how mutexes and condition variables are used.

Simple mechanism:

The statment that puts a thread to sleep should always check the condition to make sure it is satisfied before continuing,
as the thread might have been awakened due to a UNIX signal or some other reason.

```
#include <stdio.h>
#include <pthread.h>
#define MAX 1000000000                  /* how many numbers to produce */
pthread_mutex_t the_mutex;
pthread_cond_t condc, condp;            /* used for signaling */
int buffer = 0;                         /* buffer used between producer and consumer */

void *producer(void *ptr)               /* produce data */
{
    int i;

    for (i= 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex);   /* get exclusive access to buffer */
        while (buffer != 0) pthread_cond_wait(&condp, &the_mutex);
        buffer = i;                       /* put item in buffer */
        pthread_cond_signal(&condc);      /* wake up consumer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}

void *consumer(void *ptr)               /* consume data */
{
    int i;

    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex);   /* get exclusive access to buffer */
        while (buffer ==0 ) pthread_cond_wait(&condc, &the_mutex);
        buffer = 0;                       /* take item from buffer (not shown) and reinitialize */
        pthread_cond_signal(&condp);      /* wake up producer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}


int main(int argc, char **argv)
{
    pthread_t pro, con;
    pthread_mutex_init(&the_mutex, 0);
    pthread_cond_init(&condc, 0);
    pthread_cond_init(&condp, 0);
    pthread_create(&con, 0, consumer, 0);
    pthread_create(&pro, 0, producer, 0);
    pthread_join(pro, 0);
    pthread_join(con, 0);
    pthread_cond_destroy(&condc);
    pthread_cond_destroy(&condp);
    pthread_mutex_destroy(&the_mutex);
}
```

Figure 2-33. Using threads to solve the producer-consumer problem.

7. Monitors (1)

Monitor – higher-level synchronization primitive, to make the writing of correct programs simpler.

It is a collection of procedures, variables, and data-structures that are grouped together in a special kind of module or package.

The person writing the monitor does not have to be aware of how the compiler arranges for mutual exclusion. It is suficiente to know that by turning all critical regions into monitor procedures, no two processes will ever execute their critical region at the same time.

Monitors are a programming-language concept.
The compiler is in charge to implement mutual exclusion on monitor entries, not the programmer, so it is much less likely that something will go wrong.

C, Pascal, and some other languages do not have monitors.
Java support it.

```
monitor example
    integer i;
    condition c;

    procedure producer( );
    .
    .
    .
    end;

    procedure consumer( );
    .
    .
    .
    end;
end monitor;
```

Example of a monitor written in Pascal pseudocode

7. Monitors (2)

Although monitors provide an easy way to achieve mutual exclusion,
that is not enough,

we also need a way for processes to block when they can not proceed.

Again, the solution is the introduction of condition variables,

along with two operations: *wait* and *signal*.

```
monitor ProducerConsumer
        condition full, empty;
        integer count;

        procedure insert(item: integer);
        begin
                if count = N then wait(full);
                insert_item(item);
                count := count + 1;
                if count = 1 then signal(empty)
        end;

        function remove: integer;
        begin
                if count = 0 then wait(empty);
                remove = remove_item;
                count := count - 1;
                if count = N - 1 then signal(full)
        end;

        count := 0;
end monitor;

procedure producer;
begin
        while true do
        begin
                item = produce_item;
                ProducerConsumer.insert(item)
        end
end;

procedure consumer;
begin
        while true do
        begin
                item = ProducerConsumer.remove;
                consume_item(item)
        end
end;
```

Condition variables:
wait
signal

Figure 2-35. An outline of the producer-consumer problem with monitors. Only one monitor procedure at a time is active. The buffer has N slots.

7. Monitors (3)

```
#define N 100                                   /* number of slots in the buffer */
int count = 0;                                  /* number of items in the buffer */

void producer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          item = produce_item();                /* generate next item */
          if (count == N) sleep();              /* if buffer is full, go to sleep */
          insert_item(item);                    /* put item in buffer */
          count = count + 1;                    /* increment count of items in buffer */
          if (count == 1) wakeup(consumer);     /* was buffer empty? */
     }
}

void consumer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          if (count == 0) sleep();              /* if buffer is empty, got to sleep */
          item = remove_item();                 /* take item from buffer */
          count = count - 1;                    /* decrement count of items in buffer */
          if (count == N - 1) wakeup(producer); /* was buffer full? */
          consume_item(item);                   /* print item */
     }
}
```

Figure 2-27. The producer-consumer problem with a fatal race condition.

You may think that operations *wait* and *signal* looks similar to *sleep* and *wakeup*, which we saw earlier had a fatal race condition.

Well, they are very similar, but with one crucial difference: *sleep* and *wakeup* failed because while one process was trying to go to sleep, the other one was trying to wake up.

With monitors, that cannot happen.
The automatic mutual exclusion on monitor procedures guarantee that if, say, the producer inside a monitor procedure discovers that the buffer is full, it will be able to complete the *wait* operation without having to worry about the possibility that the scheduler may switch to the consumer just before the wait completes.

Java is an object-oriented language that supports user-level threads and also allow methods (procedures) to be grouped together into classes.

Synchronized methods in Java differ from classical monitors in an essential way: Java does not have built in condition variables.

Instead, it offers two procedures, *wait* and *notify*, which are equivalente of *sleep* and *wakeup* except when they are used inside synchronized methods, they are not subject to race conditions.

7. Monitors (4)

By making the mutual exclusion of critical regions automatic, monitors make parallel programming much less error than using semaphores.

Nevertheless, they too have some drawbacks, major user programming languages do not support monitors, so it is unreasonable to expect their compilers to enforce any mutual exclusion rules. In fact, how could the compiler know which procedure were the monitors and which are not?

These same languages do not have semaphores either, but adding semaphores is easy: all you need to do is add two short assembly-code routines to the library to issue the *up* and *down system calls*.

Another problem with monitors, and also with semaphores, is that they were designed for solving the mutual exclusion problem on one or more CPU, that all have access to a common memory, when we move to a distributed system consisting of multiple CPU, each with its own private memory and connected to a local area network, these primitives become inapplicable.

The conclusion is that semaphores are too low level and monitors are not usable in a few programming languages. And none of those primitives allow information exchange between machines.

8. Message Passing (1)

Message Passing – Method of interprocess communication with two primitives:

```
send(destination, &message)
receive(source, &message)
```

like semaphores, and unlike monitors, are system calls rather than language constructs (they can easily be put into library procedures).

Message-passing systems have many problems and design issues that do not arise with semaphores or with monitors, especially if the communicating processes are on diferente machines connected by a network.
Example: messages can be lost by the network.

To guard against lost messages, the sender and the receiver can agree that as soon as a message has been received, the receiver will send back a special **acknowledgement message,** if the sender has not received the acknowledgment within a certain time interval, it retransmits the message. A**uthentications**,it is also another issue in a message system.

There are also design issues that are important if the sender and the receiver are on the same machine, like performance, copying messages from one process to another is always slower than doing a semaphore or entering a monitor.

8. Message Passing (2)

Producer-Consumer problem

Solved with message passing and no shared memory.

Suming all messages are the same size and messages sent but not yet received are buffered automatically by the OS. In this solution, a total of N messages is used, analogous to the N slots in a shared-memory buffer.

If the producer works faster than the consumer, all messages will end up full, waiting for the consumer; the producer will be blocked. Waiting for na empty slot for come back. If the consumer works faster, it will happens the reverse situation.

How messages are addressed (two exemples):

- Assign each process a unique address and have messages being addressed to the processes;
- A new data structure – **mailbox**.

When mailbox are used, the address parameters in send and receive calls are mailboxes, not process.

```
#define N 100                                   /* number of slots in the buffer */

void producer(void)
{
     int item;
     message m;                                 /* message buffer */

     while (TRUE) {
          item = produce_item( );               /* generate something to put in buffer */
          receive(consumer, &m);                /* wait for an empty to arrive */
          build_message(&m, item);              /* construct a message to send */
          send(consumer, &m);                   /* send item to consumer */
     }
}


void consumer(void)
{
     int item, i;
     message m;

     for (i = 0; i < N; i++) send(producer, &m);  /* send N empties */
     while (TRUE) {
          receive(producer, &m);                /* get message containing item */
          item = extract_item(&m);              /* extract item from message */
          send(producer, &m);                   /* send back empty reply */
          consume_item(item);                   /* do something with the item */
     }
}
```

Figure 2-37. The producer-consumer problem with *N* messages.

Some applications are divided into phases abd have the rule that no process may proceed into the next phase until all processes are ready to proceed to the next phase.
When a process reach the barrier, it is blocked until all processes have reach the barrier.

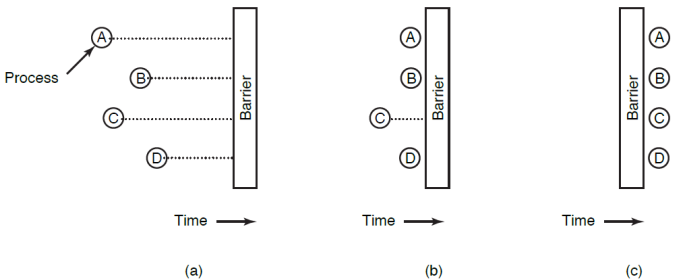


Figure 2-38. Use of a barrier. (a) Processes approaching a barrier. (b) All processes but one blocked at the barrier. (c) When the last process arrives at the barrier, all of them are let through.

10. Priority Inversion (1)

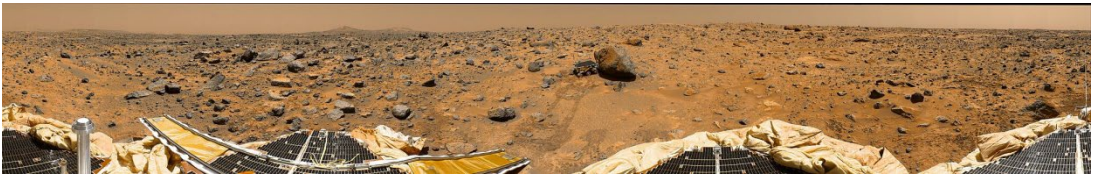
Example: Mars Pathfinder´s radio transmissions stopped sending data constantly

1st Thread - Low priority – Meteo data (mutex)
2nd Thread - High priority – Information bus management (mutex)
3rd Thread – Comunication (did not need a mutex)
Pathfinder used a shared memory called "Information bus" for passing data between its different components.



Photos and text from Mars Pathfinder – Wikipedia
Mars Pathfinder[1] was an American robotic spacecraft that landed a base station with a roving probe on Mars in 1997. It consisted of a lander, renamed the **Carl Sagan Memorial Station**, and a lightweight, 10.6 kg (23 lb) wheeled robotic Mars rover named *Sojourner*,[4] the first rover to operate outside the Earth–Moon system. **The mission terminated in 1998**.
On-board computer
The embedded computer on board the *Sojourner* rover was based around the 2 MHz[31] Intel 80C85 CPU with 512 KB of RAM and 176 KB of flash memory solid-state storage, running a cyclic executive.[32]
The computer of the *Pathfinder* lander was a Radiation Hardened IBM Risc 6000 Single Chip (Rad6000 SC) CPU with 128 MB of RAM and 6 MB of EEPROM[33][34] and its operating system was VxWorks.[35]

The mission was jeopardised by a concurrent software bug in the lander,[36] which had been found in preflight testing but was deemed a glitch and therefore given a low priority as it only occurred in certain unanticipated heavy-load conditions, and the focus was on verifying the entry and landing code. The problem, which was reproduced and corrected from Earth using a laboratory duplicate thanks to the logging and debugging functionality enabled in the flight software, was due to computer resets caused by priority inversion. No scientific or engineering data was lost after a computer reset, but all the following operations were interrupted until the next day.[37][38] Four resets occurred (on July 5, 10, 11 and 14) during the mission,[39] before patching the software on July 21 to enable priority inheritance.[40]



10. Priority Inversion (2)

There are **three different ways to solve the priority inversion problem**

-Disable all interrupts while in the critical region (simplest way).
As mentioned earlier this is not desirable for user programs.

-Priority ceiling : associate a priority with the mutex itself and assign that to the process holding it. As long as no process that needs to grab the mutex has a higher priority than the ceiling priority.

-Priority inheritance : the low priority task holding the mutex will temporarily inherit the priority of the high priority task trying to obtain it.
Again, no medium priority task will be able to preempt the task holding the mutex.

This was the technique eventually used to fix the Mar Pathfinder problem.

11. Avoiding Locks: Read-Copy-Update

The fastest locks are no locks at all.

And no locks means no risk of priority inversion, the question is whether we can allow for concurrent read and write accesses to share data strutures without locking?
In general cases the answer is NO.

However, we can allow a writer to update a data structure even though other processes are still using it. The trick is to ensure that each reader either reads the old version of the data, or the new one, but never a weird combination of old and new.
Method called RCU (Read-Copy-Update) decouples the removal and reclamation phases of the update.

Example: Removing B and D, we must be sure that there are no readers of B and D, in order to free them. How long should we wait until we are sure? 1minute? 10 min?
We must know the maximum time we need to wait- grace period – the time period in which we know that each thread to be outside its critical section at least once.

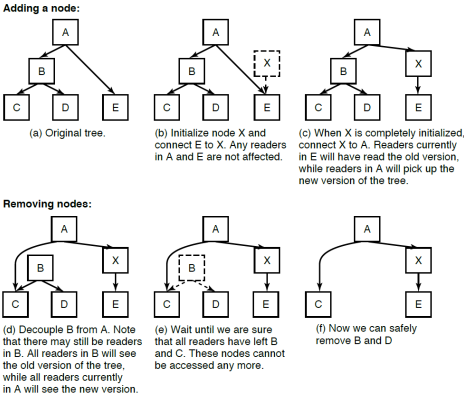


Figure 2-39. Read-Copy-Update: inserting a node in the tree and then removing a branch—all without locks.

RCU data strutures are not so common in user processes, but quite popular in operating systems kernels.

QUESTÕES?

Daniel Castro
dcastro@ualg.pt

